

5-HOUR CANDIDATE ASSIGNMENT

Computer Vision Engineering Challenge

Metric vehicle localization from a single RGB image



AI tools, pretrained models and public research are allowed.

We are evaluating how well you combine available technology, computer-vision fundamentals, experimentation and engineering judgement into an accurate, reproducible system.

TIMEBOX 5 hours Focused implementation and analysis	CORE INPUT RGB + boxes + intrinsics No depth or LiDAR at inference
CORE OUTPUT X, Z and confidence Metric location per target vehicle	FINAL PIPELINE Automatic and offline No manual image-by-image intervention

Challenge overview

Your objective

For every supplied target vehicle, estimate its lateral position x_m , forward distance z_m and a confidence score using only the RGB image, the supplied target bounding box and the supplied camera intrinsics.

TIMEBOX

5 hours

Use the time to build, compare and explain.

AI AND WEB TOOLS

Allowed

Disclose major tools, models and public code.

OBJECT DETECTION

Not required

Target bounding boxes are supplied.

FINAL INFERENCE

Local / offline

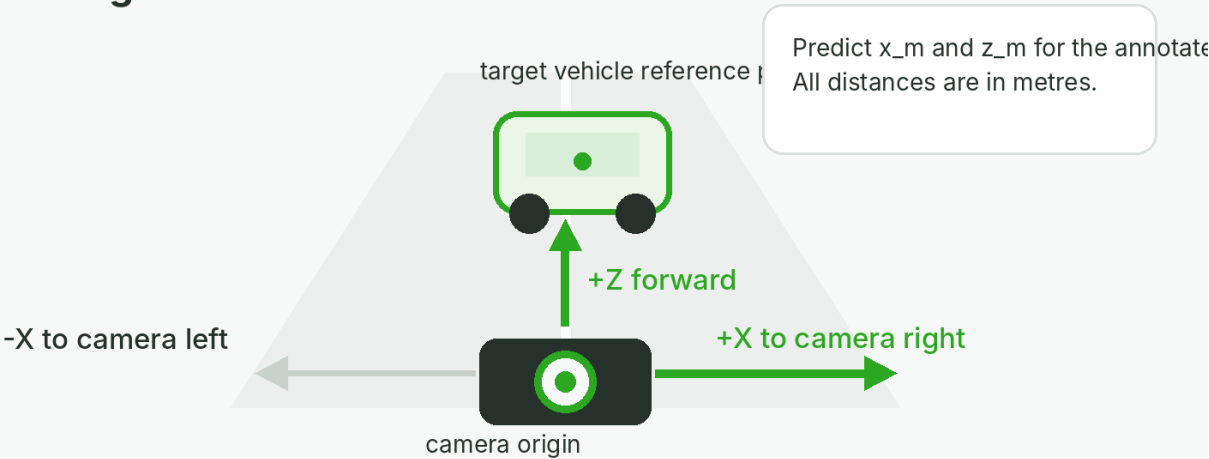
No paid, private or manual inference service.

What this challenge is designed to test

- Metric reasoning: turning image evidence into metres rather than only pixels or classes.
- Experimentation: establishing a baseline, introducing a meaningful improvement and validating it with numbers.
- Robustness: handling small, distant, occluded or edge-of-frame vehicles without fragile hard-coding.
- Engineering: producing a clean, reproducible command-line pipeline that another engineer can run.
- Judgement: understanding failure modes and knowing when the prediction should not be trusted.

Coordinate convention

Challenge coordinate convention



The challenge labels use a camera-centred coordinate system. Positive X is to the camera's right, negative X is to the left, and positive Z is forward. Predictions refer to the annotated vehicle reference point supplied by the challenge labels, not the nearest visible surface.

02

Dataset and challenge package

Source dataset: [Peking University/Baidu - Autonomous Driving on Kaggle](#). A Kaggle account may be required to access the files.

Important

This is a curated hiring challenge, not a recreation of the public Kaggle leaderboard. The hiring team will provide target boxes, development labels and a separate hidden evaluation split. Public leaderboard scores do not count.

Package structure

```
challenge/
  dev/
    images/
    targets.csv
    calibration.csv
    labels.csv
  eval/
    images/
    targets.csv
    calibration.csv
```

Input schemas

File	Required columns	Meaning
targets.csv	image_id, target_id, x1, y1, x2, y2	Top-left and bottom-right target box coordinates in the supplied image.
calibration.csv	image_id, width, height, fx, fy, cx, cy	Image dimensions and pinhole camera intrinsics for that image.
labels.csv - dev only	image_id, target_id, x_m, z_m	Ground-truth vehicle reference-point position for development and validation.

Bounding-box convention: x1/y1 is the inclusive top-left corner and x2/y2 is the exclusive bottom-right corner. All coordinates refer to the exact supplied image. If you crop or resize an image, you are responsible for updating the target coordinates and camera intrinsics consistently.

Required output

```
image_id,target_id,x_m,z_m,confidence
image_001,vehicle_001,-1.84,18.71,0.92
image_001,vehicle_002,4.12,32.84,0.71
```

- Exactly one row is required for every target_id in the evaluation manifest.
- x_m and z_m must be finite numeric values in metres.
- confidence must be between 0 and 1 and should reflect expected metric reliability, not only detector certainty.
- Missing targets, duplicate target IDs, NaN values or malformed rows may invalidate the submission.

03

Technical requirements

Build two stages, not one opaque demo

- 1 Create a working baseline that produces valid x_m , z_m and confidence predictions.
- 2 Create a materially improved approach and compare it against the baseline on the development split.
- 3 Show at least one controlled comparison or ablation that explains what produced the improvement.

What counts as a meaningful improvement?

A new signal, model, geometric constraint, aggregation strategy, calibration step or ensemble qualifies. Merely changing a threshold, prompt or learning rate without evidence does not.

You may use any combination of

- | | |
|---|---|
| <ul style="list-style-type: none"> • Pretrained metric-depth models • Camera/projective geometry • Vehicle-size or shape priors • Segmentation or masks | <ul style="list-style-type: none"> • Robust depth aggregation • Regression or lightweight fine-tuning • Ensembles and consistency checks • Uncertainty/confidence calibration |
|---|---|

Required runnable interface

```
python predict.py \
  --images ./eval/images \
  --targets ./eval/targets.csv \
  --calibration ./eval/calibration.csv \
  --output ./predictions.csv
```

An equivalent single-command interface is acceptable if it is clearly documented. The evaluator should not need to edit source files, open a notebook or manually select vehicles.

Execution expectations

- Final prediction must run automatically and locally. Development AI tools are allowed; the submitted inference pipeline must not depend on an LLM, paid API or private hosted service.
- Document all dependencies and pretrained weights. Do not rely on undeclared notebook state or files outside the repository.
- Target environment: Python 3.10+, 4 CPU cores, 16 GB RAM and, where needed, one NVIDIA T4-class GPU.
- A practical target is under 20 minutes for roughly 250 images on the stated GPU. Accuracy remains more important than micro-optimisation.

Minimum analysis

- Report MAE_Z, AbsRel_Z, P90 relative Z error and MAE_X on the supplied development labels.
- Show at least three failure cases and explain the cause, not only the symptom.
- Demonstrate that higher-confidence predictions are generally more accurate than lower-confidence predictions.

04

Deliverables and evaluation

Required deliverables

- 1 Source repository: Clean, runnable code with predict.py or an equivalent documented entry point.
- 2 predictions.csv: Predictions for every evaluation target in the required schema.
- 3 README.md: Approach, setup, assumptions, command, model/weight instructions and known limitations.
- 4 experiments.md: Baseline vs improved method, metrics, ablation/comparison and three failure cases.
- 5 visualisations/: At least 10 annotated examples showing target box, predicted X/Z, confidence and useful debug evidence.
- 6 requirements.txt: Or environment.yml / pyproject.toml with reproducible dependency versions.
- 7 SOURCES.md: External models, repositories, papers and substantial code reused, plus your modifications.
- 8 AI-use note: A short note listing the major AI tools used and how they contributed. AI use is not penalised.

Evaluation framework

Category	Points	What is assessed
Forward-distance accuracy	25	Absolute-relative and tail error on hidden targets
Lateral-position accuracy	10	Metric X error and use of camera geometry
Hard-case performance	10	Small, far, occluded and edge-of-frame vehicles
Confidence quality	10	Whether confidence predicts actual measurement risk
Geometric consistency	5	Stability under valid crop/resize/calibration changes
Engineering and reasoning	25	Reproducibility, code, experiments and failure analysis
Shortlisted live defence	15	Explain, diagnose, modify and validate the solution

The take-home submission contributes up to 85 points. The final 15 points are awarded only to shortlisted candidates during a live technical defence.

Core metrics

Forward error $MAE_Z = \text{mean}(z_{\text{pred}} - z_{\text{true}})$ $AbsRel_Z = \text{mean}(z_{\text{pred}} - z_{\text{true}} / z_{\text{true}})$	Lateral error $MAE_X = \text{mean}(x_{\text{pred}} - x_{\text{true}})$
Tail robustness P90 relative Z error: 90th percentile of relative distance error	Confidence quality High-confidence subsets should have lower error; risk-coverage behaviour is evaluated

05

Rules, integrity and submission

ALLOWED

- AI coding assistants and general-purpose LLMs
- Web search, papers and public documentation
- Open-source code and pretrained models
- Local, Colab or Kaggle compute during development
- Reasonable adaptation or fine-tuning within the timebox

NOT ALLOWED

- Manual per-image measurement or annotation of evaluation targets
- Looking up, scraping or hard-coding hidden answers
- Image-ID or perceptual-hash matching to public labels
- Final inference through a paid/private hosted API
- Sharing solutions or collaborating with other candidates

Use of public solutions

You may learn from public implementations and reuse components when licensing permits, but you must disclose them, integrate them into your own reproducible pipeline and be able to explain and modify the resulting system. An opaque copy is not a strong submission.

Submission format

Submit one ZIP archive or a private repository link through the channel specified by the recruiter. Do not include the full Kaggle dataset or large duplicate model files unless explicitly requested. Provide download instructions or scripted weight setup instead.

```
candidate_name_cv_challenge/
  README.md
  experiments.md
  SOURCES.md
  requirements.txt
  predict.py
  predictions.csv
  visualisations/
  src/
```

Final submission checklist

- ☐ The documented command runs from a clean environment.
- ☐ predictions.csv contains exactly one valid row per evaluation target.
- ☐ The baseline and improved approach are both described and compared.
- ☐ All external code, models and AI tools are disclosed.
- ☐ At least three failure cases and ten visualisations are included.
- ☐ No evaluation image was manually measured, labelled or looked up.

Frequently asked questions

Is object detection required?	No. Target bounding boxes are supplied for the core task. Detection is optional bonus work.
Is yaw/orientation required?	No. It is optional bonus work.
Is a GPU mandatory?	No. A GPU may make pretrained depth models faster, but a CPU-compatible approach is acceptable.
Can I use ChatGPT, Claude, Cursor or similar tools?	Yes. AI usage is explicitly allowed. You remain responsible for correctness, reproducibility and understanding.
What matters most?	Accurate hidden-set performance, a meaningful improvement over baseline, reliable confidence and the ability to explain failure.

Build something that works. Measure it honestly. Be ready to explain why.